

HelixTrack

HelixTrack

JIRA alternative for the free world.

Summary

HelixTrack is a comprehensive, modern, open-source alternative to JIRA (and, via its Documents extension, Confluence) — a multi-platform project-management and issue-tracking system built on a Go microservices backend with native clients for web, desktop, and mobile.

Short description

Open-source JIRA/Confluence alternative. A Go microservices backend (“HelixTrack Core”) exposes a unified REST API for project and issue tracking, plus a Confluence-style document workspace, served to native Web, Desktop, Android, and iOS clients over HTTP/3 QUIC.

Long description

HelixTrack is an open-source project-management and issue-tracking platform positioned as a free-world alternative to JIRA and Confluence — a full replacement for the two products most engineering organizations are locked into, rebuilt as software you own and can run anywhere. Its heart is **HelixTrack Core**, a REST API microservice written in Go with the Gin framework, offering full issue tracking, agile/scrum boards, team management, and a hierarchical permissions engine whose implementation is swappable between a local in-process engine and an HTTP-backed service — so the same authorization model scales from a single laptop to a distributed cluster without touching application code. Rather than sprawling a REST surface across dozens of routes, Core funnels everything through a single, action-routed `/do` endpoint with one consistent request/response envelope (action/

jwt/object/data in, errorCode/errorMessage/data out): every client speaks the same tiny contract, and adding a capability means adding an action, not a new URL to document, secure, and version. Core integrates with decoupled Authentication, Permissions, and Localization services that communicate over HTTP/3 QUIC and can run on separate machines or clusters, or be switched off entirely in test configurations. Data lives in SQLite for zero-setup development and PostgreSQL in production, and is encrypted at rest with SQLCipher (AES-256) so sensitive project data is protected on disk by default rather than as an afterthought. A **Documents V2** extension turns the tracker into a full knowledge platform: a Confluence-style workspace with spaces, pages, version control, templates, real-time WebSocket collaboration, and analytics — the wiki and the issue tracker finally living behind one backend instead of two products stitched together. Around Core sit multiple client applications: an Angular web client, a Tauri + Angular desktop client, native Android (Kotlin) and iOS (Swift) apps, plus HarmonyOS and Aurora OS clients and a screensaver — all talking to the same backend and discovering it automatically on local networks via UDP broadcast, so a fresh client finds its server without manual configuration. The client applications are maintained as separate, private repositories and are presented here at the product level only.

Why we built it

To give teams a genuinely open, self-hostable replacement for the JIRA + Confluence stack — “for the free world” — without vendor lock-in, combining enterprise-grade tracking, documents, and collaboration under one open-source license.

Why it’s a game-changer

It collapses two heavyweight commercial products — issue tracking and the wiki/documents stack — into one open, high-performance, self-hostable platform, and pairs it with something the incumbents never offered: true multi-platform *native* clients (web, desktop, Android, iOS, plus HarmonyOS and Aurora OS) all driven by a single backend contract. The unlock is ownership without compromise. An HTTP/3-everywhere, fully-decoupled microservices design and at-rest SQLCipher AES-256 encryption bring the performance and security posture usually reserved for proprietary SaaS to a system you host yourself — no seat licenses, no vendor lock-in, no data leaving your infrastructure. Teams get the JIRA-plus-Confluence experience they already know, on their own hardware, under one open-source license.

What's innovative

- Unified action-based /do API — one endpoint, one envelope, action-routed. New capabilities arrive as new actions, not new URLs, collapsing the attack surface, the client code, and the documentation burden into a single contract every platform shares.
- HTTP/3 QUIC as the *default* inter-service transport — modern low-latency, connection-resilient networking between services from day one, not bolted on later.
- A permissions engine that is swappable between a local in-process implementation and an HTTP-backed service, alongside optional, independently-deployable Auth, Permissions, and Localization services — the same authorization model whether you run one process or a cluster.
- Multi-space data isolation via a --space-root flag: each project gets its own isolated database and asset store, so tenants and projects are separated at the storage boundary rather than by query filters.
- SQLCipher AES-256 encryption at rest — sensitive project data is protected on disk transparently, by default.
- Automatic client-to-server discovery via UDP broadcast on local networks — a client finds Core with zero manual configuration.
- Documents V2, a genuine “Confluence alternative,” with optimistic-locking parallel editing, conflict detection, and full change history — real collaborative documents living behind the same backend as the tracker.

Biggest technical challenges & how we solved them

- **Six client platforms, one backend, zero contract drift.** Maintaining Web/Angular, Desktop/Tauri, Android/Kotlin, iOS/Swift, HarmonyOS, and Aurora clients normally means six divergent API integrations that rot out of sync. We collapsed that risk by making the single action-routed /do API and its fixed envelope the *only* contract — every client targets it identically — and layered UDP-broadcast service discovery on top so clients locate Core on the network without hand-configured endpoints.
- **Decoupling services without paying a latency tax.** Splitting Auth, Permissions, and Localization into independently-deployable services normally adds a network hop per call. We adopted HTTP/3 QUIC across all inter-service calls to keep those hops fast and connection-resilient, and made each service independently runnable — even fully disabled in test configs — so the decoupling is a deployment choice, not a fixed cost.

- **Confluence-grade collaboration without a lost-write mess.** Real-time multi-author editing invites conflicting writes. Documents V2 solves it with spaces/pages/versioning under optimistic locking plus explicit conflict detection, a full change history to fall back on, and real-time WebSocket sync — collaboration that stays consistent instead of silently clobbering edits.
- **Encryption at rest without killing throughput.** SQLCipher AES-256 protects data on disk but adds per-query overhead; we offset it with multi-layer caching (in-memory LRU in front of Redis in the Localization service) so hot paths like multi-language lookups stay fast while the data stays encrypted.

Tech stack

- **Go + Gin** — chosen for high-throughput, low-latency HTTP services with a single-binary deployment story; carries Core's REST API, its JWT/CORS middleware, and the action-routed /do router that fronts the whole system.
- **HTTP/3 QUIC** — chosen as the transport between Core and its Auth/Permissions/Localization services because QUIC's multiplexed, connection-migrating design cuts tail latency and survives flaky links where TCP stalls.
- **PostgreSQL (prod) / SQLite (dev)** — one relational model backing the large tracking-plus-documents schema across both engines: SQLite keeps local development zero-setup and file-based, while Postgres takes over for production scale via a dedicated production compose profile.
- **SQLCipher (AES-256)** — chosen for transparent, database-level encryption at rest so protecting sensitive project data requires no application-layer crypto and no change to how queries are written.
- **Redis** — chosen as the shared cache tier behind an in-memory LRU in the Localization service, giving a two-layer cache that keeps hot multi-language lookups fast even with encryption overhead underneath.
- **Uber Zap + Lumberjack** — chosen for structured, allocation-light logging with built-in rotation, so Core stays observable in production without unbounded log growth.
- **golang-jwt / JWT** — chosen as the stateless auth mechanism; the signed token rides in the jwt field of every /do envelope, so authentication is uniform across all clients.
- **Angular 19 (+ Material, RxJS)** — chosen for a reactive, component-driven browser client with a mature Material design system out of the box.
- **Tauri 2.0 + Rust + Angular** — chosen to ship a native desktop shell with a tiny footprint by reusing the Angular UI inside a Rust-backed webview instead of bundling a full browser runtime.

- **Kotlin (Android) / Swift + SwiftUI (iOS)** — chosen so mobile users get truly native, platform-idiomatic clients rather than a wrapped web view.
- **Docker / Docker Compose (Podman-compatible)** — chosen for reproducible, containerized deployment with /health checks wired in, and Podman compatibility so no daemon or vendor is mandated.
- **Testify (Go); Cypress/Playwright/Karma+Jasmine (clients)** — chosen for layered automated testing that covers the backend contract and the client UIs independently, matching the single-backend/many-clients architecture.

Status & honesty notes

- **Status: beta.** HelixTrack Core is a working REST API microservice; the **Documents V2** extension is documented as approximately 95% complete with a known database field-mapping issue, so it is not presented as fully shipped.
- **License: TBD.** CLAUDE.md states MIT, but the core/LICENSE file of record is Apache 2.0 — this discrepancy must be resolved before the license is stated definitively.
- Performance figures cited in the project's README (e.g. 50,000+ requests/second, sub-millisecond query times) are stated design/marketing targets, not independently published benchmarks, and are therefore omitted from the claims above.
- The client applications (Web, Desktop, Android, iOS, Aurora, HarmonyOS) live in **private** repositories and are described only at the product level.

Priority tier: Helix-primary and the flagship of the Helix-Track product line — ranked before any Server Factory projects.